

Reviewer Pack — Minimal Local Indeterminacy in Quantum Information Geometry ...

Entry 3d8d981c2574 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 04:57:56 UTC

Minimal Local Indeterminacy in Quantum Information Geometry and Resolution Proof Width

Entry ID: 3d8d981c2574 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-03 04:57:56 UTC

1. Conjecture

Field A (mathematical branch): Quantum Information Geometry **Field B** (complexity object): Resolution Proof Complexity

Statement:

For every CNF φ , the minimal local indeterminacy ($MLI\varphi$) of its associated quantum state, as measured by the maximum volume of a region in the quantum state space separated from the rest by an entanglement cost greater than some constant c , is linearly correlated with the resolution proof width $w(\varphi)$, such that $MLI\varphi = \Theta(w(\varphi))$.

Rationale (proposer's reasoning):

Quantum information geometry provides a framework for understanding quantum states as points in a geometric space. The minimal local indeterminacy captures the intricacies of these states and could potentially reveal hidden structures in the proof complexity of resolution, which has not been explored before.

Taxonomy category: QuantumInformationGeometry (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 3282226211c0ae32

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient between $MLI\varphi$ and $w(\varphi)$ exceeds 0.7 for at least 80% of seeds, with no seed showing a correlation coefficient below 0.5.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Quantum Information Geometry" AND "Resolution Proof Complexity" AND minimal local indeterminacy" - "quantum state space" AND entanglement cost AND resolution proof width" - "CNF ϕ " AND associated quantum state AND linearly correlated with resolution proof width

Top relevant hits considered: - [http://arxiv.org/abs/1505.04682v2] Geometry of quantum state space and quantum correlations - [http://arxiv.org/abs/1805.11292v1] Geometry of quantum state space and entanglement - [http://arxiv.org/abs/1804.04515v3] Quantifying entanglement in a 68-billion dimensional quantum state space

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def generate_cnf(n):
    cnf = []
    for _ in range(n):
        clause = [random.randint(1, n), -random.randint(1, n)]
        cnf.append(clause)
    return cnf

def dpll_solve(cnf, assignment={}):
    if not cnf:
        return True
    literal = next((lit for lit in range(1, len(cnf) + 1) if lit not in assignment and -lit not in assignment))
    if literal is None:
        return False

    def propagate(lit):
        new_cnf = []
        for clause in cnf:
            if lit in clause:
                continue
            if -lit in clause:
                clause.remove(-lit)
            if not clause:
                return False
            new_cnf.append(clause)
        return new_cnf, {**assignment, lit: True}
```

```

def backtrack(lit):
    new_cnf = []
    for clause in cnf:
        if -lit in clause:
            continue
        if lit in clause:
            clause.remove(lit)
            if not clause:
                return False
        new_cnf.append(clause)
    return new_cnf, {**assignment, lit: False}

if dpll_solve(propagate(lit)[0], propagate(lit)[1]):
    return True
elif dpll_solve(backtrack(-lit)[0], backtrack(-lit)[1]):
    return True
else:
    return False

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = 40
    cnf = generate_cnf(n)
    width = len(dpll_solve(cnf))

    # Placeholder for MLI calculation (not implemented)
    mli = 1.0 # Dummy value, replace with actual MLI computation

    return {
        "metric_name": "MLI",
        "metric_value": mli,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:

```

```

    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")
else:
    print(f"RESULT: INCONCLUSIVE reason=insufficient_evidence n_tested={len(results)}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4d132690.py", line 88, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4d132690.py", line 68, in run_trial
    width = len(dpll_solve(cnf))
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4d132690.py", line 56, in dpll_solve
    if dpll_solve(propagate(lit)[0], propagate(lit)[1]):
                        ^^^
NameError: name 'lit' is not defined. Did you mean: 'List'?

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete its execution to calculate the Pearson correlation coefficients for $MLI\phi$ and $w(\phi)$. As a result, we cannot verify whether the pre-registered support condition is met. | next: Re-run the test with proper error handling to ensure that it completes execution and produces the necessary data for calculating the Pearson correlation coefficients.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13988
2	preregistration	ollama_remote	glm4:latest	0	0	9318
3	novelty	ollama_remote	glm4:latest	0	0	8540
4	novelty	ollama_remote	glm4:latest	0	0	9433
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13381
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20734
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11371

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18193
9	judge	ollama_remote	glm4:latest	0	0	13179

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 118137 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/3d8d981c2574.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/3d8d981c2574.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/3d8d981c2574.tar.gz (if generated)